КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет комп'ютерних наук та кібернетики

ЗВІТ
до лабораторної роботи № 1

з дисципліни «Сучасні технології баз даних»

Тема: «Система управління бібліотекою з додатковими послугами»

Київ — 2026

1. Мета та завдання роботи

Мета роботи — спроєктувати та реалізувати реляційну базу даних для системи управління бібліотекою з додатковими послугами, застосувавши ключі, зв’язки, обмеження цілісності, індекси, Soft Delete, збережені процедури, функції, тригери, представлення та роботу з БД із C#.

- розробити схему БД із не менше ніж 15 сутностями;
- для щонайменше 6 сутностей реалізувати Soft Delete, дату останньої зміни та користувача, який її виконав;
- реалізувати модель у PostgreSQL;
- створити процедури, функції, тригери та VIEW (мінімум 10 об’єктів);
- створити не менше 6 індексів щонайменше двох типів;
- реалізувати Repository + Unit of Work у C#, використовуючи процедури та VIEW.

2. Проєктування бази даних

Предметна область охоплює облік користувачів, читачів, працівників, книг і фізичних примірників, видач, бронювань, штрафів та додаткових послуг. Початкову схему створено в dbdiagram.io, після чого її експортовано до PostgreSQL.

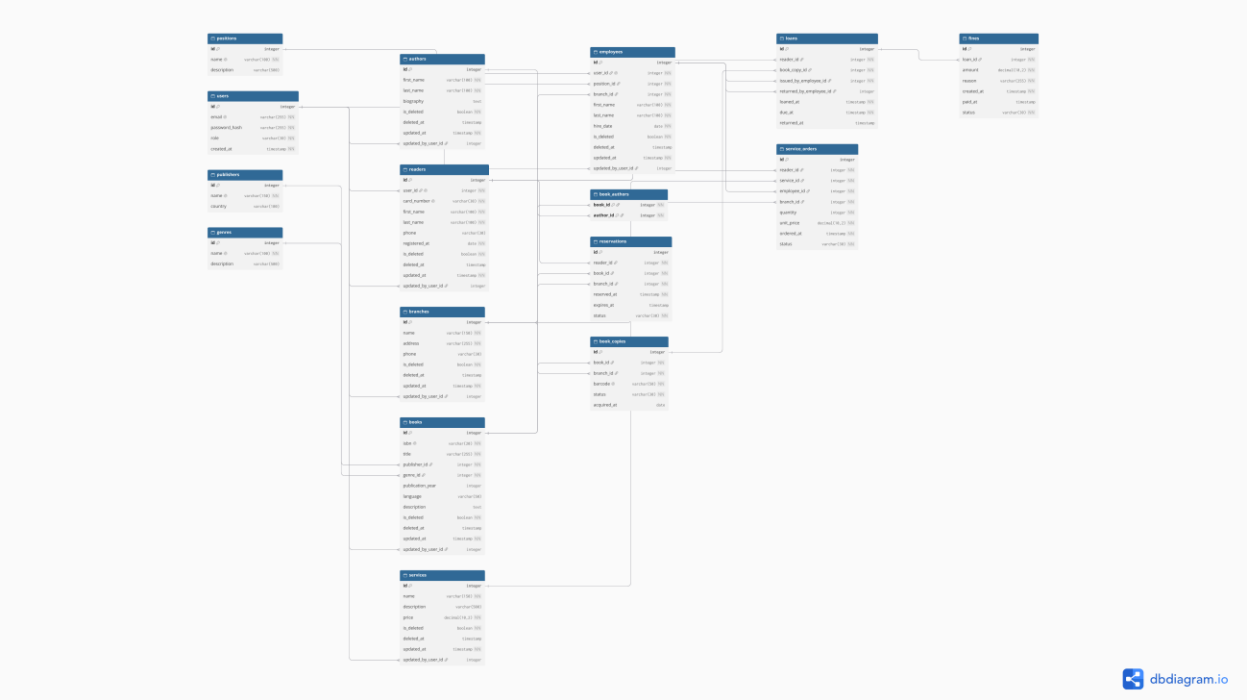


Рисунок 1 – Схема бази даних бібліотеки

2.1. Сутності

1. users	9. books

2. readers	10. book_authors
3. positions	11. book_copies
4. branches	12. loans
5. employees	13. reservations
6. publishers	14. services
7. authors	15. service_orders
8. genres	16. fines

2.2. Основні зв’язки

Ключові зв’язки: users пов’язана з readers та employees; positions і branches — з employees; publishers та genres — з books; authors і books мають зв’язок M:N через book_authors; books має багато book_copies; loans пов’язує читача, конкретний примірник та працівника; reservations пов’язує читача, книгу й філію; service_orders — читача, послугу, працівника та філію; одна видача може мати кілька штрафів.

3. Реалізація структури в PostgreSQL

У PostgreSQL створено 16 таблиць. Для більшості сутностей використано сурогатний первинний ключ id типу INTEGER GENERATED BY DEFAULT AS IDENTITY. Для book_authors застосовано складений первинний ключ (book_id, author_id).

Характерний приклад створення таблиць:

```
CREATE TABLE books (
    id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    isbn VARCHAR(20) NOT NULL,
    title VARCHAR(255) NOT NULL,
    publisher_id INTEGER NOT NULL,
    genre_id INTEGER NOT NULL,
    publication_year INTEGER,
    is_deleted BOOLEAN NOT NULL DEFAULT false,
    deleted_at TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT now(),
    updated_by_user_id INTEGER
);

CREATE TABLE book_copies (
    id INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    book_id INTEGER NOT NULL,
    branch_id INTEGER NOT NULL,
    barcode VARCHAR(50) NOT NULL,
    status VARCHAR(30) NOT NULL,
    acquired_at DATE
);
```

Зовнішні ключі задано окремими ALTER TABLE ... ADD CONSTRAINT. Наприклад, loans посилається на readers, book_copies та employees. Так забезпечується посилальна цілісність.

```
ALTER TABLE loans
ADD CONSTRAINT fk_loans_reader
FOREIGN KEY (reader_id) REFERENCES readers(id);

ALTER TABLE loans
ADD CONSTRAINT fk_loans_book_copy
FOREIGN KEY (book_copy_id) REFERENCES book_copies(id);
```

4. Обмеження цілісності та індекси

Крім PRIMARY KEY, FOREIGN KEY, NOT NULL та UNIQUE, додано CHECK-обмеження для ролей, статусів, цін, кількості та дат. Це не дозволяє зберігати некоректні стани.

```
ALTER TABLE users
ADD CONSTRAINT chk_users_role
CHECK (role IN ('reader', 'employee', 'admin'));

ALTER TABLE book_copies
ADD CONSTRAINT chk_book_copies_status
CHECK (status IN ('available', 'borrowed', 'damaged', 'lost', 'repair'));

ALTER TABLE service_orders
ADD CONSTRAINT chk_service_orders_quantity
CHECK (quantity > 0);
```

Створено більше шести індексів. Використано прості, складені, унікальні та неунікальні індекси. PostgreSQL також автоматично створює індекси для PRIMARY KEY та UNIQUE.

```
CREATE UNIQUE INDEX ux_users_email ON users(email);
CREATE INDEX ix_authors_last_name ON authors(last_name);
CREATE INDEX ix_books_title ON books(title);
CREATE UNIQUE INDEX ux_book_copies_barcode ON book_copies(barcode);
CREATE INDEX ix_book_copies_book_branch ON book_copies(book_id,
branch_id);
CREATE INDEX ix_loans_reader_returned ON loans(reader_id, returned_at);
```

5. Soft Delete та аудит змін

Soft Delete реалізовано для readers, branches, employees, authors, books і services. Запис не видаляється фізично: is_deleted стає true, a deleted_at фіксує час логічного видалення. Поля updated_at та updated_by_user_id зберігають інформацію про останню зміну.

Тригерна функція аудиту:

```
CREATE OR REPLACE FUNCTION set_audit_fields()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
   v_user_id INTEGER;
BEGIN
   NEW.updated_at := CURRENT_TIMESTAMP;
   v_user_id := NULLIF(current_setting('app.user_id', true), '')::INTEGER;

   IF v_user_id IS NOT NULL THEN
      NEW.updated_by_user_id := v_user_id;
   END IF;

   RETURN NEW;
END;
$$;
```

Ця функція підключена BEFORE UPDATE тригерами до шести таблиць. Для кожної з них створено окрему процедуру Soft Delete. Приклад:

```
CREATE OR REPLACE PROCEDURE soft_delete_service(
   p_service_id INTEGER,
   p_user_id INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
   PERFORM set_config('app.user_id', p_user_id::text, true);

   UPDATE services
   SET is_deleted = true,
      deleted_at = CURRENT_TIMESTAMP
   WHERE id = p_service_id
     AND is_deleted = false;
END;
$$;
```

6. Процедури, функції та тригери

6.1. Видача і повернення книг

Процедура borrow_book створює запис у loans. Перед вставкою тригер перевіряє активність читача та працівника, наявність примірника і його статус. Після успішної вставки інший тригер автоматично змінює book_copies.status на borrowed.

```
CREATE OR REPLACE PROCEDURE borrow_book(
    p_reader_id INTEGER,
    p_book_copy_id INTEGER,
    p_employee_id INTEGER,
    p_due_at TIMESTAMP
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_due_at <= CURRENT_TIMESTAMP THEN
        RAISE EXCEPTION 'Дата повернення повинна бути пізніше поточної
дати';
    END IF;

    INSERT INTO loans(reader_id, book_copy_id, issued_by_employee_id,
loaned_at, due_at)
    VALUES(p_reader_id, p_book_copy_id, p_employee_id,
CURRENT_TIMESTAMP, p_due_at);
END;
$$;
```

Процедура return_book заповнює returned_by_employee_id і returned_at. Тригер, що реагує на заповнення returned_at, автоматично повертає примірнику статус available.

6.2. Користувацькі функції

Створено дві звичайні функції: calculate_overdue_fine — обчислює штраф за прострочення; get_available_copies_count — повертає кількість доступних примірників книги.

```
SELECT calculate_overdue_fine(2);
SELECT get_available_copies_count(1);
```

6.3. Тригери

У БД використано 6 аудит-тригерів та 3 тригери для loans: перевірка перед видачею, встановлення статусу borrowed після INSERT і повернення статусу available після заповнення returned_at.

7. Представлення (VIEW)

Для роботи з готовими наборами даних створено 5 представлень. Саме через них виконується частина запитів із C#.

VIEW	Призначення
v_available_books	доступні примірники книг з авторами, жанром, видавцем і філією
v_active_loans	активні видачі та кількість днів прострочення
v_reader_loan_history	історія видач читача і сума штрафів
v_active_services	послуги, що не видалені логічно
v_service_order_details	деталі замовлень послуг і розрахована загальна вартість

Приклад:

```
CREATE OR REPLACE VIEW v_active_services AS
SELECT id, name, description, price
FROM services
WHERE is_deleted = false;
```

8. Тестове наповнення та перевірка

Базу заповнено невеликою кількістю тестових записів для всіх основних сутностей: користувачів, читачів, працівників, книг, примірників, видач, бронювань, послуг, замовлень і штрафів. Тестові INSERT-запити у звіті повністю не наводяться, оскільки їх призначення — лише перевірка логіки.

Перевірено: створення всіх 16 таблиць і зв’язків; роботу CHECK-обмежень та індексів; Soft Delete; автоматичне заповнення updated_at/updated_by_user_id; видачу і повернення книги; роботу VIEW та користувацьких функцій. Також перевірено, що одна видача може мати кілька штрафів.

9. Робота з БД із C#: Repository + Unit of Work

Створено консольний застосунок .NET із пакетом Npgsql. Для доступу до БД використано два репозиторії: BookRepository і LoanRepository. BookRepository читає v_available_books, а LoanRepository читає v_active_loans і викликає процедури borrow_book та return_book.

Структура:

Program
  -> UnitOfWork
      -> BookRepository -> v_available_books
      -> LoanRepository -> v_active_loans
                  -> borrow_book()
                  -> return_book()

UnitOfWork зберігає одне спільне NpgsqlConnection для репозиторіїв і підтримує BeginTransaction, Commit та Rollback. Під час перевірки всередині транзакції було створено видачу, після чого ROLLBACK повністю скасував зміни.

```
await unitOfWork.BeginTransactionAsync();

await unitOfWork.Loans.BorrowBookAsync(
    readerId,
    bookCopyId,
    employeeId,
    dueAt);

await unitOfWork.RollbackAsync();
```

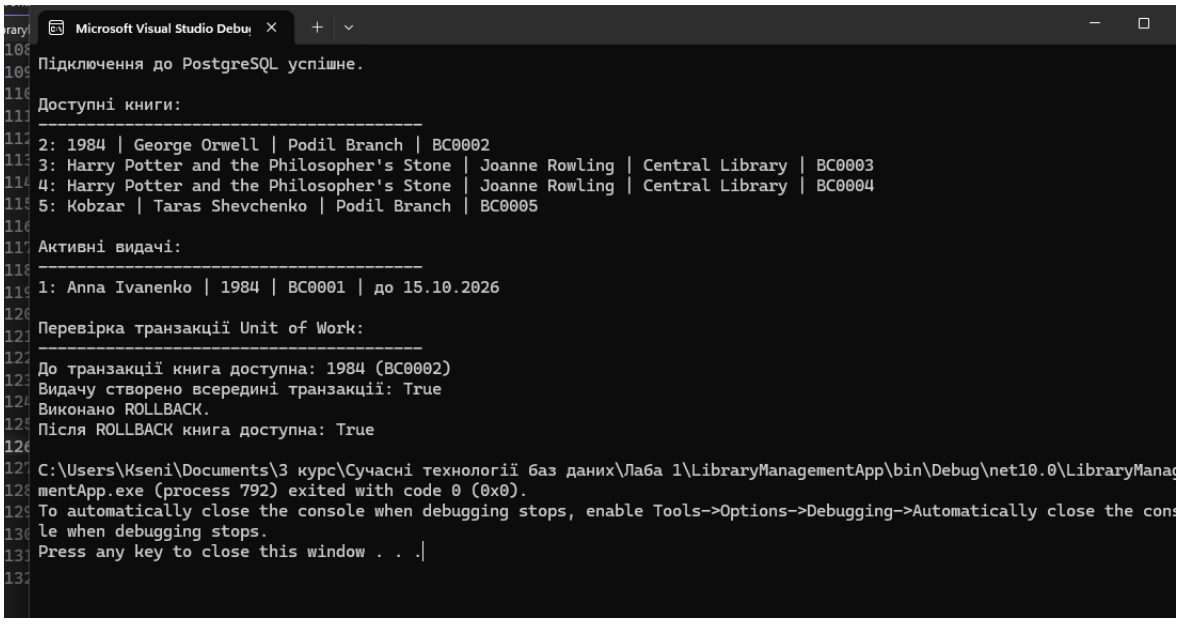


Рисунок 2 – Перевірка Repository + Unit of Work у C#

10. Виконання вимог лабораторної роботи

Вимога	Реалізація	Стан
Сутності	16 таблиць	виконано

Soft Delete	6 сутностей	виконано
Аудит змін	updated_at + updated_by_user_id	виконано
СУБД	PostgreSQL	виконано
Процедури/функції/тригери/VIEW	більше 10 об’єктів	виконано
Індекси	більше 6; різні типи	виконано
Repository + Unit of Work	C# + Npgsql + транзакції	виконано
Доступ із коду	через VIEW і процедури	виконано

11. Висновок

У лабораторній роботі спроєктовано та реалізовано реляційну базу даних системи управління бібліотекою з додатковими послугами. Створено 16 взаємопов’язаних сутностей, обмеження цілісності та індекси, реалізовано Soft Delete й аудит змін, збережені процедури, функції, тригери та VIEW. У C# реалізовано Repository + Unit of Work із транзакціями; доступ до бізнес-операцій здійснюється через процедури PostgreSQL, а читання підготовлених даних — через VIEW. Усі вимоги лабораторної роботи виконано.

Повний SQL-скрипт розгортання бази даних з усіма таблицями, обмеженнями, індексами, процедурами, функціями, тригерами, VIEW та тестовими даними збережено окремо у файлі LibraryDb_full.sql.